The overall connection

C source → preprocessor → compiler → assembly → assembler → object files → linker → executable machine instructions. Types determine how bit patterns are interpreted. The ISA defines what instructions mean. Processor control and datapath circuits execute them, changing registers, memory, and the program counter (PC).

C: compilation, types, and functions

- **Declaration:** gives a name and type. **Definition:** also creates storage or a function body. Multiple compatible declarations may refer to one definition.
- **Header (.h):** publishes declarations, prototypes, types, and macros. `#include` is processed before compilation. Use include guards. Compile separate `.c` files to `.o` files with `gcc -c`; the linker combines them.
- `gcc -Wall` enables warnings; `-o name` sets the executable name. `main` returns an integer; returning 0 conventionally signals success.
- C uses declared types but permits conversions. A conversion may discard information (for example, floating value to integer). Uninitialized automatic variables have indeterminate values.
- In the course's common model, `short/int/long` occupy 2/4/8 bytes and `float/double` 4/8 bytes. Actual C type widths depend on the implementation.
- Function arguments are passed by **value**: parameters receive copies. Passing a pointer copies the address, allowing a function to modify the pointed-to object.
- A function prototype declares parameter and return types. A local `static` retains its value between calls; file-scope `static` restricts linkage to that source file.
- `&&` has higher precedence than `||`. Parentheses make intended evaluation clear. A `do-while` loop executes its body before testing the condition.
- For `scanf`, use `%d` for `int`, `%f` for `float`, and `%lf` for `double`; pass addresses. `%p` prints a pointer.

Pointers, arrays, strings, structs

- `&x` obtains the address of `x`. `T *p` declares a pointer to `T`; `*p` accesses the pointed-to object. `p+1` advances by `sizeof(T)` bytes.
- Never dereference an uninitialized, invalid, null, or dangling pointer. **Aliasing** means two pointers can refer to one object. A dangling pointer refers to an object whose lifetime has ended.
- Array elements have the same type and occupy contiguous storage. In a function parameter, `int a[8]` is adjusted to `int *a`; the 8 does not enforce the caller's array size.
- `int a[5]={1,2,3}` sets remaining elements to zero. An uninitialized local array does not have guaranteed zero values.
- A C string is a character array terminated by `'\0'`. Reserve a byte for the terminator. `fgets` can retain a newline. `scanf("%s",...)` stops at whitespace and should use a width limit.
- A struct groups fields of potentially different types. Struct assignment copies its value. `p->field` means `(*p).field`. Passing a struct by value copies it; passing an array argument permits modification of its elements.

Stored-program architecture and memory

CPU = registers + ALU + control. Memory holds both instructions and data; I/O communicates with the outside. A bus connects CPU and memory. Registers are small, fast storage inside the CPU. **Load:** memory → register. **Store:** register → memory. If too many values are live, the compiler **spills** some to memory and reloads them later.

In the lecture's 32-bit MIPS model, a word is 32 bits = 4 bytes. Memory is **byte-addressed**; aligned word addresses satisfy $a \bmod 4 = 0$. The next aligned word begins at $a+4$, not $a+1$. A k-bit byte address identifies 2^k byte locations.

Bits and positional number systems

One bit has two states; n bits have 2^n patterns. Encoding M distinct values requires at least $\lceil \log_2 M \rceil$ bits. Positional value is

$$(d_k \cdots d_0 . d_{-1} \cdots)_r = \sum_i d_i r^i.$$

For an integer, repeatedly divide by the target radix and read remainders backward. For a fraction, repeatedly multiply by the radix and read integer parts forward. Group binary digits in threes for octal and fours for hexadecimal. The same bits can represent different things: `01000110` is integer 70 or ASCII character `F`.

Signed integer representations

First decide whether the same n-bit pattern is *unsigned*, sign magnitude, one's complement, two's complement, or excess-K. The arithmetic and interpretation depend on that choice. MSB means most significant (leftmost) bit. An out-of-range mathematical answer is overflow even if an n-bit result pattern can still be written.

Unsigned binary. Range: 0 to $2^n - 1$. Add normally; an outgoing carry from bit $n-1$ means unsigned overflow. Retaining only n bits computes the sum modulo 2^n. Subtraction with $A < B$ wraps modulo 2^n if kept in n bits. There is no representation of a negative value.

Sign magnitude: range, negation, addition

Interpretation and range: MSB is sign (0 positive, 1 negative); the remaining $n-1$ bits give magnitude. Range is $-(2^{n-1}-1)$ to $2^{n-1}-1$. Both $+0$ and -0 exist.

Negate: flip only the sign bit. This also switches the two zero encodings. **Add:** if signs match, add magnitudes and keep the common sign. If signs differ, subtract the smaller magnitude from the larger and use the sign of the larger-magnitude operand. Equal opposite magnitudes give zero. **Subtract:** negate the second operand, then add using these rules.

Overflow: for same-sign addition, the magnitude sum exceeds $2^{n-1}-1$. Opposite-sign addition cannot overflow, because its magnitude is no larger than the larger input magnitude. A carry from a raw addition of complete sign-magnitude bit strings is *not* the arithmetic rule.

One's complement: range, negation, addition

Interpretation and range: nonnegative values use ordinary binary; a negative value is the bitwise complement of its positive magnitude. Range is $-(2^{n-1}-1)$ to $2^{n-1}-1$. There are two zeros: all 0s is $+0$, all 1s is -0.

Negate: invert every bit, $x \mapsto \overline{x}$. For a positive integer x, its negative pattern has unsigned value $2^n - 1 - x$. For example, in 8 bits, $+12 = 00001100$ and $-12 = 11110011$.

Add two numbers: (1) add the n-bit patterns as ordinary binary; (2) if a carry leaves the MSB, add that carry back into the low bit (*end-around carry*); (3) interpret the n-bit result and check overflow. Example in 4 bits: $5 + (-3) : 0101 + 1100 = 1\,0001 \to 0010 = 2$. **Subtract:** complement the second operand, then use one's-complement addition with end-around carry.

Overflow: after end-around carry, adding two positive values must not produce a negative result, and adding two negative values must not produce a positive result. Equivalently, the true sum must remain in $[-(2^{n-1}-1), 2^{n-1}-1]$. Opposite-sign addition cannot overflow. Handle the two zero patterns as zero when reasoning about signs.

Two's complement: range, negation, addition

Interpretation and range: nonnegative values use ordinary binary; a pattern with MSB 1 represents its unsigned value minus 2^n. Range is -2^{n-1} to $2^{n-1}-1$. There is one zero and one extra negative number.

Negate: invert all bits and add 1: $-x = \overline{x}+1 \bmod 2^n$. For a positive integer x, the negative pattern has unsigned value $2^n - x$. In 8 bits, $+12 = 00001100$ and $-12 = 11110100$. The minimum value -2^{n-1} cannot be negated within n signed bits: $+2^{n-1}$ is out of range.

Add two numbers: add the n-bit patterns, retain the low n bits, and discard the carry out of the MSB. Then check *signed* overflow. In 4 bits, $5+(-3): 0101+1101 = 1\,0010 \to 0010 = 2$. **Subtract:** $A-B = A+\overline{B}+1$, using the same n-bit adder and overflow test.

Overflow: same-sign addition produces an opposite-sign result, or equivalently the carry into the sign bit differs from the carry out. Two opposite-sign operands cannot overflow on addition. A final carry out alone is *not* signed overflow: $1111+0001 = 1\,0000$ is $-1+1=0$, without overflow. For $A-B$, overflow occurs if A and B have different signs and the result sign differs from A's sign.

Excess or biased representation

Encode: store $x+K$ as an unsigned n-bit pattern. **Decode:** $x = \text{stored} - K$. Range is $-K$ to 2^n-1-K. There is no dedicated sign bit. Ordinary binary addition of two stored patterns does *not* directly give the biased encoding of their sum: decode, operate, then re-encode (or correct the bias). IEEE 754 uses a **bias-127** exponent field for 32 bits.

Fixed-point range and spacing

With f fractional bits, a signed fixed-point value is its stored integer multiplied by 2^{-f}. Spacing is 2^{-f}. For an n-bit two's-complement pattern:

$$\text{range} = \left[-2^{n-1-f}, (2^{n-1}-1)2^{-f}\right].$$

Example: $n=8, f=2$ gives range $[-32, 31.75]$ in steps of 0.25. For an *unsigned* fixed-point pattern the range is $[0, (2^n-1)2^{-f}]$. If a sign-magnitude or one's-complement integer is scaled by 2^{-f}, its range is $[-(2^{n-1}-1)2^{-f}, (2^{n-1}-1)2^{-f}]$. All these fixed-point formats have spacing 2^{-f} between successive nonzero values. More fractional bits improve resolution but reduce range at fixed total width. **Note:** the spacing is 2^{-f}, not 2^f.

IEEE 754 single precision (binary32)

A 32-bit floating-point pattern has three fields:

$$\underbrace{s}_{1\ \text{sign bit}} \;\Big|\; \underbrace{E}_{8\ \text{exponent bits}} \;\Big|\; \underbrace{F}_{23\ \text{fraction bits}}.$$

$s=0$ is positive and $s=1$ is negative. Interpret E and F as unsigned integers. The exponent has bias 127; the fraction is $f = F/2^{23}$. Classify the pattern by its *stored* exponent before applying the normal-number formula.

Normal finite numbers ($1 \le E \le 254$):

$$x = (-1)^s(1+F/2^{23})\,2^{E-127} = (-1)^s(1.f)_2\,2^{E-127}.$$

The leading 1 is implicit, so 23 stored fraction bits give 24 significant binary digits. The actual exponent ranges from -126 to $+127$. To encode a nonzero normal number, write it as $\pm 1.b_1b_2\ldots_2 \times 2^e$, store $E = e+127$, and store the bits after the leading 1 in F, rounding if needed. Example: $6.5_{10} = 110.1_2 = 1.101_2\,2^2$; $s=0$, $E = 129 = 10000001_2$, $F = 101$ followed by 20 zeros. Its 32-bit hex pattern is `0x40D00000`.

binary32 special patterns

- **Signed zero:** $E=0, F=0$. $s=0$ gives $+0$ (`0x00000000`); $s=1$ gives -0 (`0x80000000`). Both compare equal numerically, although their bit patterns and behavior in some operations differ.
- **Subnormal:** $E=0, F\neq 0$. There is *no* implicit leading 1. The effective exponent is -126:
$$x = (-1)^s(F/2^{23})\,2^{-126} = (-1)^s F\,2^{-149}.$$
Subnormals fill the gap between zero and the smallest normal number. Their spacing is constant at 2^{-149}.
- **Infinity:** $E=255, F=0$. Sign chooses $+\infty$ (`0x7F800000`) or $-\infty$ (`0xFF800000`). Overflow can produce infinity, depending on the rounding mode.
- **NaN (not a number):** $E=255, F\neq 0$. It denotes an undefined numerical result, such as $0/0$ or $\infty-\infty$. Fraction bits can carry a payload; its sign does not determine an ordinary positive/negative number. `0x7FC00000` is a common quiet-NaN pattern.

Boundary values (positive): smallest subnormal 2^{-149}; largest subnormal $(2^{23}-1)2^{-149}$; smallest normal 2^{-126}; largest finite $(2-2^{-23})2^{127} \approx 3.4028235 \times 10^{38}$. A nonzero result too small for normal representation can become subnormal or round to zero. The gap between adjacent normal values with actual exponent e is 2^{e-23}, so absolute precision changes with scale. Around 1, the upward gap is 2^{-23}; binary32 provides roughly 7 decimal significant digits.

IEEE 754 rounding and quick decoding

Rounding modes: nearest with ties to even (default), toward zero, toward $+\infty$, and toward $-\infty$. For a value cut after the last stored fraction bit, the **guard bit** is the first discarded bit, the **round bit** is the next, and the **sticky bit** is the OR of all later discarded bits. They help decide whether the stored fraction must increase. Rounding after each operation means $(a+b)+c$ can differ from $a+(b+c)$.

Exam procedure: split the 32 bits into $s \mid E \mid F$; check $E=0$ and $E=255$ first. For $1 \le E \le 254$, restore the hidden leading 1 and use exponent $E-127$. For $E=0, F\neq 0$, use leading 0 and exponent -126. For $E=255$, distinguish infinity from NaN using $F=0$ versus $F\neq 0$. A 64-bit double instead uses 1 sign, 11 exponent bits (bias 1023), and 52 fraction bits.

Range quick reference: all representations

For an n-bit *integer*, the first value below is the most negative representable value and the second is the largest positive. Both $+1$ and -1 are the nonzero values closest to zero whenever the format includes both signs.

Integer format	Representable numeric range
Unsigned	0 to 2^n-1
Sign magnitude	$-(2^{n-1}-1)$ to $2^{n-1}-1$; two zeros
One's complement	$-(2^{n-1}-1)$ to $2^{n-1}-1$; two zeros
Two's complement	-2^{n-1} to $2^{n-1}-1$; one zero
Excess-K	$-K$ to 2^n-1-K; e.g. 4-bit excess-8 is -8 to 7

For *fixed point* with f fractional bits, multiply each integer endpoint above by 2^{-f}. Thus unsigned spans 0 to $(2^n-1)2^{-f}$; sign magnitude and one's complement span $\pm(2^{n-1}-1)2^{-f}$; two's complement spans -2^{n-1-f} to $(2^{n-1}-1)2^{-f}$. The smallest nonzero magnitude is 2^{-f} when the format can represent it. For excess-K fixed point, scale $[-K, 2^n-1-K]$ by 2^{-f}.

For *IEEE 754 binary32*, the finite extremes are $\pm M_{32}$, where $M_{32} = (2-2^{-23})2^{127} \approx 3.4028235 \times 10^{38}$. The least positive *normal* is 2^{-126}; the least positive nonzero *subnormal* is 2^{-149}. The negative value closest to zero is -2^{-149}, while the most negative finite value is $-M_{32}$. Infinity and NaN are special patterns, not finite range endpoints.

For *IEEE 754 binary64*, the corresponding values are $M_{64} = (2-2^{-52})2^{1023} \approx 1.7976931348623157 \times 10^{308}$, minimum positive normal 2^{-1022}, and minimum positive subnormal 2^{-1074}. Its finite extremes are $\pm M_{64}$; the negative value closest to zero is -2^{-1074}. Floating-point values are discrete and unevenly spaced, so these limits do not mean every real number between them is representable.

ISA and MIPS design

The instruction set architecture (ISA) is the visible contract between software and hardware: operations, registers, data, addressing, and encoding. CISC favors richer instructions; RISC uses simpler, regular instructions composed by software.

Operand organization may be stack, accumulator, general-purpose register, or memory-memory. MIPS uses a **register-register / load-store** design: ALU operations use registers, while `lw/sw` access memory. Addressing modes in these notes: register, immediate constant, and displacement (base + offset). ISA design chooses storage, addressing, operation set, instruction fields/length, and binary encoding.

MIPS registers and common operations

MIPS has 32 general-purpose registers of 32 bits each. A register index needs 5 bits. `$zero` always reads 0; the PC is separate. Most ALU instructions have destination first: `add rd,rs,rt`, `sub`, `and`, `or`, `xor`, `nor`, `slt`. `slt` writes 1 if the first source is less than the second, else 0.

`sll/srl` shift logically; a 5-bit shift field represents 0–31 positions. `nor rd,rs,$zero` implements bitwise NOT. `move` is a pseudoinstruction.

`addi rt,rs,imm` uses a signed 16-bit immediate: -32768 to 32767, sign-extended to 32 bits. `andi/ori/xori` zero-extend their 16-bit masks. `lui` puts an immediate in the upper 16 bits and sets the lower 16 to zero. Thus an arithmetic immediate and a logical mask with the same bits can be extended differently.

MIPS memory and control flow

`lw rt,offset(rs)` computes effective byte address $R[\text{rs}] + \text{signext (offset)}$, then loads one word into `rt`. `sw rt,offset(rs)` stores `rt` at that address. The offset counts **bytes**, and word accesses require 4-byte alignment in this model.

`beq rs,rt,label` branches if equal; `bne` if not equal. `slt` followed by a branch expresses an inequality. `j label` jumps unconditionally. A label names an address; it is not an instruction.

MIPS instruction formats: every instruction is 32 bits

R format:

$$\text{opcode}_6 \mid \text{rs}_5 \mid \text{rt}_5 \mid \text{rd}_5 \mid \text{shamt}_5 \mid \text{funct}_6.$$

Typical arithmetic uses two register sources and `rd` destination; `funct` chooses the operation. Shifts use `shamt` and usually `rt` as data source.

I format:

$$\text{opcode}_6 \mid \text{rs}_5 \mid \text{rt}_5 \mid \text{immediate}_{16}.$$

`rt` is a destination for `addi/lw`, but source data for `sw` and a comparison operand for branches. The immediate's meaning depends on the opcode.

J format:

$$\text{opcode}_6 \mid \text{target}_{26}.$$

Instructions are four aligned bytes, so sequential next PC is PC + 4. A branch immediate counts **words**:

$$\text{branch target} = \text{PC} + 4 + \left(\text{signext } (\text{imm}_{16}) \ll 2 \right).$$

A jump uses pseudo-direct addressing:

$$\text{jump target} = \{(\text{PC} + 4)_{31:28}, \text{target}_{25:0}, 00\}.$$

Its upper four bits come from PC + 4, limiting the jump to the corresponding 256 MiB region.

Datapath components and five stages

The datapath contains the PC, instruction memory, register file (32 registers of 32 bits; two read ports and one write port), ALU, dedicated adders, data memory, and multiplexers. The control path selects routes, operations, and enabled writes.

1. **Fetch:** read instruction memory at PC; compute PC + 4.
2. **Decode/register read:** interpret opcode/fields, read `rs` and `rt`, extend immediate, generate controls.
3. **Execute:** ALU performs arithmetic/logic, calculates load/store address, or compares branch operands. A separate adder calculates the branch target.
4. **Memory:** `lw` reads data memory; `sw` writes it; other supported instructions do neither.
5. **Writeback:** write an ALU result or memory value when `RegWrite=1`.

These are conceptual stages, not necessarily five clock cycles.

ALU and two-level control

The ALU performs arithmetic and logical work; each bit's carry-out feeds the next bit's carry-in. Its control bits are `Ainvert`, `Binvert`, and a 2-bit `Operation` selector. For subtraction, invert B and set the least-significant carry-in to 1: $A - B = A + \overline{B} + 1$. The 4-bit `ALUcontrol` value selects the ALU's operation. The ALU also produces `isZero`, which is 1 if its result is zero; this is a *status output*, not an instruction-decoder output.

Two-level decoding: instruction opcode → main control signals and 2-bit `ALUop`; then **`ALUop`** + **`funct`** (for R type) → 4-bit `ALUcontrol` → actual ALU action. `ALUop` classifies how to determine the operation; `ALUcontrol` directly drives the ALU. The `funct` field is ignored for `lw`, `sw`, and `beq`.

What each control signal means

Mux selectors (choose a data path):

- **RegDst:** 0 chooses instruction bits [20:16], `rt`, as the register-file destination; 1 chooses bits [15:11], `rd`. Its value is irrelevant when `RegWrite=0`.
- **ALUSrc:** 0 supplies the ALU's second input from register Read Data 2; 1 supplies the sign-extended 16-bit immediate in this simplified datapath. `beq` needs 0 to compare two registers.
- **MemToReg:** 0 writes the ALU result to a register; 1 writes data read from memory. It matters only if `RegWrite=1`. The mux selects *what* to write; `RegWrite` decides *whether* a write occurs.
- **PCSrc:** 0 chooses the sequential next PC, PC + 4; 1 chooses the branch target PC + 4 + (signext (imm) ≪ 2).

Write and memory enables:

- **RegWrite:** 1 enables a write to the register file at the active clock edge; 0 suppresses it. `lw` and supported R-type ALU instructions write a register; `sw/beq` do not. Writes to MIPS `$zero` are ignored. PC updating is separate.
- **MemRead:** 1 reads data memory at the ALU-computed address (used by `lw`). **MemWrite:** 1 writes register Read Data 2 to data memory at that address (used by `sw`). Both are 0 for R type and `beq`; both 1 is unused in this datapath.

Operation and branch signals:

- **Branch:** main control sets this to 1 for `beq` in the lecture's branch circuit. It identifies a conditional branch; it does not by itself say whether the branch is taken.
- **ALUop (2 bits):** 00 means add for `lw/sw` address calculation; 01 means subtract for `beq` comparison; 10 means use R-type `funct`; 11 is unused here.

- **ALUcontrol (4 bits):** final ALU selector generated from `ALUop` and, when needed, the 6-bit `funct`. The course table uses 0010 = add, 0110 = subtract, 0000 = AND, 0001 = OR, 0111 = set-on-less-than.
- **isZero:** ALU output, 1 iff ALU result is zero. For `beq`, subtracting $R[\text{rs}] - R[\text{rt}]$ makes it 1 when the operands are equal.

ALUop and ALUcontrol lookup

Instruction	ALUop	funct	ALUcontrol / action
`lw, sw`	00	ignored	0010 / add base + offset
`beq`	01	ignored	0110 / subtract, test zero
`add`	10	100000	0010 / add
`sub`	10	100010	0110 / subtract
`and`	10	100100	0000 / bitwise AND
`or`	10	100101	0001 / bitwise OR
`slt`	10	101010	0111 / set less than

Branch decision: the opcode decoder supplies `Branch`; the ALU supplies `isZero`; a separate adder supplies the branch address. For the `beq`-only design, $\boxed{\text{PCSrc} = \text{Branch} \wedge \text{isZero}}$. For `bne`, the comparison condition must instead detect *not equal*.

Main-control settings by instruction

Type	Dst	Src	M2R	RW	MR	MW	Br	ALUop
R type	1	0	0	1	0	0	0	10
`lw`	0	1	1	1	1	0	0	00
`sw`	X	1	X	0	0	1	0	00
`beq`	X	0	X	0	0	0	1	01

Dst = RegDst; Src = ALUSrc; M2R = MemToReg; RW = RegWrite; MR = MemRead; MW = MemWrite; Br = Branch. X = don't care because the corresponding output is not used for that instruction.

Trace each instruction through the datapath

R-type ALU operation (for example, `add rd,rs,rt`): Fetch the instruction, read $R[\text{rs}]$ and $R[\text{rt}]$, then set `ALUSrc=0`. `ALUop=10` tells the ALU-control unit to inspect `funct`; the final `ALUcontrol` selects the operation. Select `rd` with `RegDst=1` and the ALU result with `MemToReg=0`; `RegWrite=1` writes the result. Data memory is inactive. Sequential execution selects $\text{PC} + 4$.

`lw rt,offset(rs)`: Read base $R[\text{rs}]$, sign-extend the 16-bit byte offset, and set `ALUSrc=1`. With `ALUop=00`, the ALU adds base and offset to form the effective address. `MemRead=1` obtains the word. Select `rt` (`RegDst=0`) and memory data (`MemToReg=1`); `RegWrite=1` saves it in the register file.

`sw rt,offset(rs)`: The ALU forms the same base-plus-offset address as for `lw`. The register file also reads $R[\text{rt}]$, which supplies data memory's Write Data input. Set `MemWrite=1`, `MemRead=0`, and `RegWrite=0`. `RegDst` and `MemToReg` are don't-cares because there is no register writeback.

`beq rs,rt,label`: Read two registers and set `ALUSrc=0`. `ALUop=01` selects subtraction; `isZero=1` means the values were equal. In parallel, a dedicated adder forms $\text{PC} + 4 + (\text{signext (imm)} \ll 2)$. The opcode sets `Branch=1`; thus `PCSrc=isZero` for this instruction. No data-memory access or general-purpose register write occurs.

A quick way to derive control values

First ask *where the second ALU operand comes from* (`ALUSrc`). Next ask *which ALU operation is needed* (`ALUop` then `ALUcontrol`). Ask whether data memory is read or written (`MemRead`, `MemWrite`). If a register is written, choose its number (`RegDst`) and writeback value (`MemToReg`), then enable `RegWrite`. Finally decide whether the next PC is sequential or a taken branch (`Branch`, `isZero`, `PCSrc`). A mux select is a don't-care whenever its output is not used.

Clocking and implementation choices

During a clock period, current state feeds combinational logic; enabled state elements capture results at the active edge. The PC supplies the current instruction address while next-PC logic computes its replacement.

Single cycle: each instruction completes in one clock period; the period must fit the longest path, typically `lw` (instruction memory → register read → ALU → data memory → register writeback).

Multicycle: execution steps occupy separate cycles. Intermediate results need storage; a controller tracks the step. The period can be shorter, but instructions need variable numbers of cycles.

Pipelining: different instructions occupy different stages at once, separated by pipeline registers. Once filled, ideal throughput approaches one completed instruction per cycle; dependencies and hazards can cause stalls or flushes. A faster clock alone does not guarantee a faster program.

Concept links and exam pitfalls

- C array indexing → element size → byte offset → base+offset effective address → `lw/sw`. For a 4-byte `int` array, element i begins $4i$ bytes after the base.
- Signed integer → two's-complement bit pattern → sign extension of an arithmetic immediate → ALU addition/subtraction and overflow check.
- Floating-point value → sign, biased exponent, fraction → finite range and relative precision → rounding after an operation.
- C expression → compiler temporaries → processor registers. Register pressure can spill values to memory, introducing extra loads and stores.
- C `if` or loop → `slt` or a branch comparison → encoded offset → target-address adder and next-PC multiplexer.
- Branch offsets count *words* and are shifted left two places; `lw/sw` offsets count *bytes*. Both ultimately form byte addresses.
- In I format, `rt` is not always a destination. `sw` uses it as source data, and `beq` compares it.
- Carry-out is not the same as signed two's-complement overflow. `RegWrite` does not control PC updating. `isZero` comes from the ALU, whereas `Branch` comes from decoding.
- The simplified datapath and control table cover `lw`, `sw`, `beq`, and R-type `add/sub/and/or/slt`. `bne`, jumps, shifts, immediate arithmetic, and other instructions require corresponding datapath or control extensions.